

Get started

- Overview
- Quickstart
- Models
- Pricing
- SDKs and CLI
- Latest: GPT-5.5
- Prompt guidance

Core concepts

- Text generation
- Code generation
- Images and vision
- Audio and speech
- Structured output
- Function calling
- Responses API
- Using tools

Agents SDK

- Overview
- Quickstart
- Agent definitions
- Models and providers
- Running agents
- Sandbox agents
- Orchestration
- Guardrails
- Results and state
- Integrations and observability
- Evaluate agent workflows
- Voice agents
- Agent Builder

Tools

- Web search
- MCP and Connectors
- Skills
- Shell
- Computer use
- File search and retrieval
- Tool search
- More tools

Run and scale

- Conversation state
- Background mode
- Streaming
- WebSocket mode
- Webhooks
- File inputs
- Context management
- Prompting
- Reasoning

Evaluation

- Getting started
- Working with evals
- Prompt optimizer

Working with evals

Test and improve model outputs through evaluations.

Copy Page

Evaluations (often called **evals**) test model outputs to ensure they meet style and content criteria that you specify. Writing evals to understand how your LLM applications are performing against your expectations, especially when upgrading or trying new models, is an essential component to building reliable applications.

In this guide, we will focus on **configuring evals programmatically using the [Evals API](#)**. If you prefer, you can also configure evals **[in the OpenAI dashboard](#)**.

If you're new to evaluations, or want a more iterative environment to experiment in as you build your eval, consider trying [Datasets](#) instead.

Broadly, there are three steps to build and run evals for your LLM application.

- Describe the task to be done as an eval
- Run your eval with test inputs (a prompt and input data)
- Analyze the results, then iterate and improve on your prompt

This process is somewhat similar to behavior-driven development (BDD), where you begin by specifying how the system should behave before implementing and testing the system. Let's see how we would complete each of the steps above using the [Evals API](#).

Create an eval for a task

Creating an eval begins by describing a task to be done by a model. Let's say that we would like to use a model to classify the contents of IT support tickets into one of three categories: `Hardware`, `Software`, or `Other`.

To implement this use case, you can use either the [Chat Completions API](#) or the [Responses API](#). Both examples below combine a [developer message](#) with a user message containing the text of a support ticket.

```
Categorize IT support ticketspython

1 from openai import OpenAI
2 client = OpenAI()
3
4 instructions = """
5 You are an expert in categorizing IT support tickets. Given the support
6 ticket below, categorize the request into one of "Hardware", "Software",
7 or "Other". Respond with only one of those words.
8 """
9
10 ticket = "My monitor won't turn on - help!"
11
12 response = client.responses.create(
13     model="gpt-4.1",
14     input=[
15         {"role": "developer", "content": instructions},
16         {"role": "user", "content": ticket},
17     ],
18 )
19
20 print(response.output_text)
```

Let's set up an eval to test this behavior [via API](#). An eval needs two key ingredients:

- `data_source_config`: A schema for the test data you will use along with the eval.
- `testing_criteria`: The [graders](#) that determine if the model output is correct.

```
Create an evalcurl

1 curl https://api.openai.com/v1/evals \
2   -H "Authorization: Bearer $OPENAI_API_KEY" \
3   -H "Content-Type: application/json" \
4   -d '{
5     "name": "IT Ticket Categorization",
6     "data_source_config": {
7       "type": "custom",
8       "item_schema": {
9         "type": "object",
10        "properties": {
11          "ticket_text": { "type": "string" },
12          "correct_label": { "type": "string" }
13        },
14        "required": ["ticket_text", "correct_label"]
15      },
16      "include_sample_schema": true
17    },
18    "testing_criteria": [
19      {
20        "type": "string_check",
21        "name": "Match output to human label",
```

Responses

Create an eval for a task

Test a prompt with your eval

Analyze the results

Next steps

Copy Page

```

22         "input": "{{ sample.output_text }}",
23         "operation": "eq",
24         "reference": "{{ item.correct_label }}"
25     }
26 ]
27 },'

```

> Explanation: `data_source_config` parameter

> Explanation: `testing_criteria` parameter

After creating the eval, it will be assigned a UUID that you will need to address it later when kicking off a run.

```

1 {
2   "object": "eval",
3   "id": "eval_67e321d23b54819096e6bfe140161184",
4   "data_source_config": {
5     "type": "custom",
6     "schema": { ... omitted for brevity... }
7   },
8   "testing_criteria": [
9     {
10      "name": "Match output to human label",
11      "id": "Match output to human label-c4fdf789-2fa5-407f-8a41-a6f4f9afd482",
12      "type": "string_check",
13      "input": "{{ sample.output_text }}",
14      "reference": "{{ item.correct_label }}",
15      "operation": "eq"
16    }
17  ],
18  "name": "IT Ticket Categorization",
19  "created_at": 1742938578,
20  "metadata": {}
21 }

```

Now that we've created an eval that describes the desired behavior of our application, let's test a prompt with a set of test data.

Test a prompt with your eval

Now that we have defined how we want our app to behave in an eval, let's construct a prompt that reliably generates the correct output for a representative sample of test data.

Uploading test data

There are several ways to provide test data for eval runs, but it may be convenient to upload a [JSONL](#) file that contains data in the schema we specified when we created our eval. A sample JSONL file that conforms to the schema we set up is below:

```

1 { "item": { "ticket_text": "My monitor won't turn on!", "correct_label": "Hardware" } }
2 { "item": { "ticket_text": "I'm in vim and I can't quit!", "correct_label": "Software" } }
3 { "item": { "ticket_text": "Best restaurants in Cleveland?", "correct_label": "Other" } }

```

This data set contains both test inputs and ground truth labels to compare model outputs against.

Next, let's upload our test data file to the OpenAI platform so we can reference it later. You can upload files [in the dashboard here](#), but it's possible to [upload files via API](#) as well. The samples below assume you are running the command in a directory where you saved the sample JSON data above to a file called `tickets.jsonl`:

```

Upload a test data file
curl https://api.openai.com/v1/files \
-H "Authorization: Bearer $OPENAI_API_KEY" \
-F purpose="evals" \
-F file="@tickets.jsonl"

```

When you upload the file, make note of the unique `id` property in the response payload (also available in the UI if you uploaded via the browser) - we will need to reference that value later:

```

1 {
2   "object": "file",
3   "id": "file-CwHg45Fo7YXwkWRPukLNHw",
4   "purpose": "evals",
5   "filename": "tickets.jsonl",
6   "bytes": 208,
7   "created_at": 1742834798,
8   "expires_at": null,
9   "status": "processed",
10  "status_details": null
11 }

```

Creating an eval run

With our test data in place, let's evaluate a prompt and see how it performs against our test criteria. Via API, we can do this by [creating an eval run](#).

Make sure to replace `YOUR_EVAL_ID` and `YOUR_FILE_ID` with the unique IDs of the eval configuration and test data files you created in the steps above.

```

Create an eval run curl ↕
1 curl https://api.openai.com/v1/evals/YOUR_EVAL_ID/runs \
2   -H "Authorization: Bearer $OPENAI_API_KEY" \
3   -H "Content-Type: application/json" \
4   -d '{
5     "name": "Categorization text run",
6     "data_source": {
7       "type": "responses",
8       "model": "gpt-4.1",
9       "input_messages": {
10        "type": "template",
11        "template": [
12          {"role": "developer", "content": "You are an expert in categorizing IT support tickets. Gi
13          {"role": "user", "content": "{{ item.ticket_text }}"}}
14        ]
15      },
16      "source": { "type": "file_id", "id": "YOUR_FILE_ID" }
17    }
18  }'

```

When we create the run, we set up a prompt using either a [Chat Completions](#) messages array or a [Responses](#) input. This prompt is used to generate a model response for every line of test data in your data set. We can use the double curly brace syntax to template in the dynamic variable `item.ticket_text`, which is drawn from the current test data item.

If the eval run is successfully created, you'll receive an API response that looks like this:

```

1 {
2   "object": "eval.run",
3   "id": "evalrun_67e44c73eb6481909f79a457749222c7",
4   "eval_id": "eval_67e44c5bec81909704be0318146157",
5   "report_url": "https://platform.openai.com/evaluation/evals/abc123",
6   "status": "queued",
7   "model": "gpt-4.1",
8   "name": "Categorization text run",
9   "created_at": 1743015028,
10  "result_counts": { ... },
11  "per_model_usage": null,
12  "per_testing_criteria_results": null,
13  "data_source": {
14    "type": "responses",
15    "source": {
16      "type": "file_id",
17      "id": "file-J7MoX9ToHXp2TutMEeYnwj"
18    },
19    "input_messages": {
20      "type": "template",
21      "template": [
22        {
23          "type": "message",
24          "role": "developer",
25          "content": {
26            "type": "input_text",
27            "text": "You are an expert in..."
28          }
29        },
30        {
31          "type": "message",
32          "role": "user",
33          "content": {
34            "type": "input_text",
35            "text": "{{item.ticket_text}}"
36          }
37        }
38      ]
39    },
40    "model": "gpt-4.1",
41    "sampling_params": null
42  },
43  "error": null,
44  "metadata": {}
45 }

```

Your eval run has now been queued, and it will execute asynchronously as it processes every row in your data set, generating responses for testing with the prompt and model we specified.

Analyze the results

To receive updates when a run succeeds, fails, or is canceled, create a webhook endpoint and subscribe to the `eval.run.succeeded`, `eval.run.failed`, and `eval.run.canceled` events. See the [webhooks guide](#) for more details.

Depending on the size of your dataset, the eval run may take some time to complete. You can view current status in the dashboard, but you can also [fetch the current status of an eval run via API](#):

```

Retrieve eval run status curl ↕
1 curl https://api.openai.com/v1/evals/YOUR_EVAL_ID/runs/YOUR_RUN_ID \
2   -H "Authorization: Bearer $OPENAI_API_KEY" \
3   -H "Content-Type: application/json"

```

You'll need the UUID of both your eval and eval run to fetch its status. When you do, you'll see eval run data that looks like this:

```

1 {

```

```

2      "object": "eval.run",
3      "id": "evalrun_67e44c73eb6481909f79a457749222c7",
4      "eval_id": "eval_67e44c5bec81909704be0318146157",
5      "report_url": "https://platform.openai.com/evaluation/evals/xxx",
6      "status": "completed",
7      "model": "gpt-4.1",
8      "name": "Categorization text run",
9      "created_at": 1743015028,
10     "result_counts": {
11       "total": 3,
12       "errored": 0,
13       "failed": 0,
14       "passed": 3
15     },
16     "per_model_usage": [
17       {
18         "model_name": "gpt-4o-2024-08-06",
19         "invocation_count": 3,
20         "prompt_tokens": 166,
21         "completion_tokens": 6,
22         "total_tokens": 172,
23         "cached_tokens": 0
24       }
25     ],
26     "per_testing_criteria_results": [
27       {
28         "testing_criteria": "Match output to human label-40d67441-5000-4754-ab8c-181c125803ce",
29         "passed": 3,
30         "failed": 0
31       }
32     ],
33     "data_source": {
34       "type": "responses",
35       "source": {
36         "type": "file_id",
37         "id": "file-J7Mox9ToHXp2TutMEeYnwj"
38       },
39       "input_messages": {
40         "type": "template",
41         "template": [
42           {
43             "type": "message",
44             "role": "developer",
45             "content": {
46               "type": "input_text",
47               "text": "You are an expert in categorizing IT support tickets. Given the support ticket"
48             }
49           },
50           {
51             "type": "message",
52             "role": "user",
53             "content": {
54               "type": "input_text",
55               "text": "{{item.ticket_text}}"
56             }
57           }
58         ]
59       },
60       "model": "gpt-4.1",
61       "sampling_params": null
62     },
63     "error": null,
64     "metadata": {}
65   }

```

The API response contains granular information about test criteria results, API usage for generating model responses, and a `report_url` property that takes you to a page in the dashboard where you can explore the results visually.

In our simple test, the model reliably generated the content we wanted for a small test case sample. In reality, you will often have to run your eval with more criteria, different prompts, and different data sets. But the process above gives you all the tools you need to build robust evals for your LLM apps!

Next steps

Now you know how to create and run evals via API, and using the dashboard! Here are a few other resources that may be useful to you as you continue to improve your model results.



Cookbook: Detecting prompt regressions

Keep tabs on the performance of your prompts as you iterate on them.



Cookbook: Bulk model and prompt experimentation

Compare the results of many different prompts and models at once.



Cookbook: Monitoring stored completions

Examine stored completions to test for prompt regressions.



Fine-tuning

Improve a model's ability to generate responses tailored to your use case.



Model distillation

Learn how to distill large model results to smaller, cheaper, and faster models.

